

Tutorial 4: WebGL Tutorial - Part I

ITCS 3120 Computer Graphics

Fall 2020

Revision 11/9/2020 4:41:15 PM

Professor Zachary Wartell

Copyright September 2015, Zachary Wartell @ University of North Carolina at Charlotte
All Rights Reserved.

Learning Objectives

- Learn how WebGL integrates with HTML5 and the HTML5 Canvas
- Exercise basic prior knowledge of HTML5 and JavaScript
- Learn how WebGL integrates with JavaScript
- Exercise basic prior knowledge of using Developer Tools with the Chrome web browser
- Exercise basic prior knowledge of JavaScript, HTML5 Canvas, and mouse and keyboard events
- Learn the basics of the WebGL shading language.

1. Overview

This assignment requires you have a copy of:

- [~\$30] WebGL Programming Guide: Interactive 3D Graphics Programming with WebGL
 - Suggested: Kindle Edition from Amazon (~\$30) (Kindle readers app's are available free for all operating systems).
- [Free] [WebGL v1.0 Quick Reference Card.pdf](#)
 - familiarize yourself with this handy “card”
- Additional programming resources are also found on the course Canvas pages under “Programming References”
 - Under “Programming References” for JavaScript there is a link to an advertisement free on-line guide to JS plus a detailed discussion of which aspects of JS will be most relevant to this course.

For this project, in the WebGL Book you will read Chapter 1, Chapter 2 and part of Chapter 3. You will download roughly a dozen example programs. Chapter 2 walks you through all the fundamental components of the source code of these example programs and introduces how WebGL integrates with HTML5 and JavaScript and introduces the fundamentals of the WebGL API and the WebGL shading language. As you read through the Chapter 2 examples, this document will instruct you to make explicit, trivial modifications for each of the example programs and to use good Git practices for incrementally committing your work. The Chapter 3 exercises will cover just the first two sections of Chapter 3.

Revision 11/9/2020 4:40:00 PM

This project is just a tutorial. Later projects will require you to develop your own code.

2. Git Commit Protocol and Rubric

In the detailed instructions below, you are required to make git commits at specific points using specific commit messages. This is a required part of the assignment. It also motivates good git practices.

1. For a given exercise that includes specific git commit instructions, if there is not at least 1 git commit message associated with that exercise, 1 point will be subtracted for "poor software development practices" for that exercise.

The student's commit message should be a verbatim copy of the commit message given in the exercise's instructions. (You can simply cut & paste the messages).

(The TA will allow a slight leniency for a few non-verbatim messages)

Note: If you accidentally commit with a missing or bad message, you can correct the message using git commit –amend option (see <https://git-scm.com/book/en/v2/Git-Basics-Undoing-Things>)

Finally, extra commit's with messages are perfectly fine and common practice, such for fixing other types of coding mistakes, etc.

2. For a given exercise, if only the original example code is committed with no change & there is no evidence from the student (documentation, notes, etc) of trying to solve the exercise, then only 2% of that exercise's total points will be given for that exercise (round upward for fractional credit).

The complete grading Rubric is available on Canvas.

3. Project Organization

Your working directory structure and your git repository structure must be exactly as described below. Otherwise, the TA will not be able to run your code. (*Reason:* When you read Chapter 2 you will see the WebGL book example code assumes the book's library code resides in a specific relative path).

1. Create the following directory structure on your computer.

```
$ mkdir ITCS_3120
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cd ITCS_3120
```

2. Git clone the WebGL book's example code from:

https://cci-git.uncc.edu/UNCC_Graphics_Third_Party_Libraries/WebGL_Programming_Guide.git

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$git clone https://cci-
git.uncc.edu/UNCC_Graphics_Third_Party_Libraries/WebGL_Programming_Guide.git

Cloning into 'WebGL_Programming_Guide'...
remote: Counting objects: 224, done.
remote: Compressing objects: 100% (72/72), done.
Receiving objects: 100% (224/224), 455.88 KiB | 0 bytes/s, done.bjects: 100% (224/224)
```



3. Create a Git repository:

https://cci-git.uncc.edu/your_userid/3120_T4_WebGL

- a. Make sure the repository is **Private**
- b. Give the Dr. Wartell and the TA Reporter access (see the earlier Git Tutorial for instructions).

4. Clone the repository:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ git clone https://cci-git.uncc.edu/your_userid/3120_T4_webGL.git
T4_WebGL
Cloning into 'T4_WebGL'...
warning: You appear to have cloned an empty repository.
Checking connectivity... done.
```

5. Copy the common WebGL book libraries to your ITCS_3120 directory:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp -r WebGL_Programming_Guide/lib .
```

6. Verify your directory structure:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ ls -la
total 12
drwxr-xr-x  1 zwartell Administ  4096 Sep 17 17:40 .
drwxr-xr-x 46 zwartell Administ 12288 Sep 17 17:02 ..
drwxr-xr-x  3 zwartell Administ    0 Sep 17 17:34 T4_WebGL
drwxr-xr-x  1 zwartell Administ  4096 Sep 17 17:09 WebGL_Programming_Guide
drwxr-xr-x  6 zwartell Administ  4096 Sep 17 17:40 lib
```

7. Copy the Chapter 2 example code to T4_WebGL:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp -r WebGL_Programming_Guide/ch02/* T4_webGL/

zwartell@CCIWD-435B-1 ~/ITCS_3120
$ ls T4_webGL/ [verify result]
ClickedPoints.html  ColoredPoints.html  DrawRectangle.html  HelloCanvas.html
HelloPoint1.html    HelloPoint2.html
ClickedPoints.js    ColoredPoints.js    DrawRectangle.js    HelloCanvas.js
HelloPoint1.js      HelloPoint2.js
```

8. Git add, commit and push this initial code with the indicated message:

```
$ cd T4_webGL/

zwartell@CCIWD-435B-1 ~/ITCS_3120/T4_WebGL (master)
$ git add .

zwartell@CCIWD-435B-1 ~/ITCS_3120/T4_WebGL (master)
$ git commit -m "-imported webGL Chapter 2 code"
[master (root-commit) 9d7325a] -imported WebGL Chapter 2 code
12 files changed, 396 insertions(+)
create mode 100644 ClickedPoints.html
create mode 100644 ClickedPoints.js

$ git push origin master
Counting objects: 14, done.
Delta compression using up to 24 threads.
Compressing objects: 100% (14/14), done.
Writing objects: 100% (14/14), 2.98 KiB | 0 bytes/s, done.
```

4. Introduction to WebGL + HTML5 + JavaScript

- 1) Web Browser and Browser Development Tools:

This tutorial and future class tutorials will make use of the “Developer Tools” available in web browsers. All web browsers contain developer tools. However, this tutorial and future class tutorials will assume you are using the Chrome web browser and Chrome’s developer tools. Further, the TA will test your assignments using the Chrome web browser.

In your previous tutorial you already installed Chrome and used some of its developer tools.

- 2) Read the WebGL books online instructions in the sample code directory which discuss setting up your browser to allow access to local files. Two copies of these instructions are available:

- a) in your ITCS_3120 directory:

```
"ITCS_3120/WebGL_Programming_Guide/ITCS_4120_Extra/docs/Downloads - WebGL
Programming Guide.html"
```

b) at the WebGL Guide's www:

<https://sites.google.com/site/webglbook/home/downloads>

3) Skim over WebGL book Appendix H: which has additional tips for Web Browser Settings for WebGL.

4) Read WebGL Book Chapter 1. In the book, any figures with example code whose URL is shown something like:

`file:///C:/samples/ch09/RotateObject.html`

can be found in your corresponding ITCS_3120 working directory:

`ITCS_3120\WebGL_Programming_Guide\ch09\RotateObject.html`

5) Read WebGL Book Chapter 2 (but read one section at a time, as describe below).

While reading, open up each example program in a WebGL capable web browser. Use the browser's Development Tools to examine all the HTML, JavaScript and WebGL shader code in the browser script viewer windows, while you read Chapter 2's description of the code.

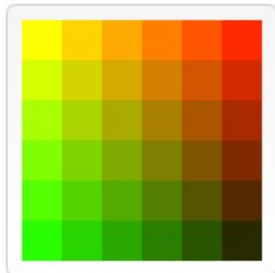
a) Exercise:

Read Chapter 2 Section "What Is a Canvas?". Study the code.

Modify your copy of DrawRectangle.html (i.e. in ITCS_3120/T4_WebGL/DrawRectangle.html) as follows:

(Note, you should draw all of the following on the one canvas)

- i) Change the size of the original rectangle to be 1.5 times bigger.
- ii) Change the color of the original rectangle to any other color.
- iii) Add second rectangle of any color and size and position you choose.
- iv) Use a pair of for loops generate a 4 by 4 array 16 small rectangles each with a different color. Below is a 6 x 6 array example. You use any set of colors you want.



JavaScript Hint: The WebGL book barely introduces JavaScript. Read the following section of the recommended JavaScript reference regarding JS loops:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Guide/Loops_and_iteration

- v) **Git** add and commit this updated code using the message:

"-exercise - What Is a Canvas?"

Tip: It is also good practice to git push your changes occasionally so they are safely backed up on the server.

- b) **Exercise**:

Read Chapter 2 Section “The World’s Shortest WebGL Program: Clear Drawing Area.”

Modify your copy of HelloCanvas.html (i.e. ITCS_3120/T4_WebGL/HelloCanvas.html) as follows:

- i) Add an additional console log message when the GLContext is successfully created.
- ii) Change the Canvas size to 800 by 800
- iii) Modify the clear color is red.
- iv) **Git** add all the updated code. git commit all the changed files with a single commit using the message:

"-exercise - Hello World"

- v) Modify the code so that the clear color is randomly chosen every time the page is loaded (i.e. if the user hit browser “Reload” button, she should see a different, random color each time).

JavaScript Hint: The WebGL book barely introduces JavaScript. See:

https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/Math/random

- vi) **Git** add, commit with a single commit using the message:

"-added random coloring"

- c) **Exercise**:

Open HelloPoint1.html Read Chapter 2 Section ”Draw a Point (Version 1)”

- i) Modify the program to draw the point
 - (1) in a different color.
 - (2) at a different position
 - (3) in a different size

- ii) **Git** add, commit with a single commit using the message:

"-exercise - Draw a Point"

- d) Exercise:

- i) Copy the Chapter 2 extra example code to T4_WebGL:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp -r "WebGL_Programming_Guide/ITCS_4120_Extra/ch02/." T4_WebGL/
```

- ii) **Git** add and commit this code with message:

"-added extras"

- iii) Read "Appendix A. No Need to Swap Buffers in WebGL". Pay attention to the discussion on `alert` and `confirm`; note the wording:

"This implies that if you call methods that return control to the browser, such as `alert()` or `confirm()`, the browser **may** then display the contents of the color buffer to the screen."

- iv) Open `T4_WebGL/HelloCanvas1.html` in a browser and study the js code.

- v) Question 1: Before executing it, what do you expect to see?

- vi) Execute the code.

- vii) Question 2: What do you see?

- viii) Change `enableAlert` to `true` and save the file. Execute the code again.

- ix) Question 3: What do you see?

- x) Question 4:

Regarding Question 3, you may see different behavior in different browsers and even different versions of the same browser (Chrome's behavior changed between 2015 and 2016!). In some browsers' the "**may**" in the earlier quote, turns out to be "does not" and in others it turns out to be "does".

Assume, that in a particular browser in the second case (ix), you do see all 4 different colors. For such as browser, answer the following question:

Using you knowledge of how double buffering works (discussed in class) and using the discussion in WebGL book Appendix A, explain in writing why, in the first case (vii), you only see a black square, but in the second case (ix) you see 4 different

colored squares -- even though in the first case there is a delay loop that should allow you to see the other 4 colors, each for a brief moment.

Use a diagram (neat, scanned hand sketched pictures are ok; or use your preferred drawing tool) to help explain your answer.

- xi) Put your written answers to Questions 1 through 4 in a document and save the document to file (use either PDF format or MS Word format):

ITCS_3120/T4_WebGL/T4_WebGL_Report.pdf

- xii) **Git** add and commit with message:

"-answered swap buffers questions"

- e) Exercise:

Open `HelloPoint2.html` in a browser and read Chapter 2 Section "Draw a Point (Version 2)".

- i) Modify the `vertexAttrib` function call to use the 2D version of the call so you only have to pass x,y coordinates.
- ii) Modify the location of the displayed point.
- iii) Modify the size and color of the displayed point.
- iv) As described in the Chapter modify the point size code to use another attribute shader variable.

- v) **Git** add and commit with message:

"-exercise - Draw Point V2"

- f) Exercise:

Open `ClickedPoint.html` in a browser and read Chapter 2 Section "Draw a Point with a Mouse Click"

- i) As your read through section "Experimenting with the Sample Program" make all the indicate changes to the program.

- ii) **Git** add and commit with message:

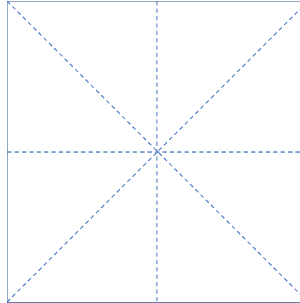
"-exercise - ClickedPoint"

- g) Exercise:

Open `ColoredPoints.html` in your browser read Chapter 2 Section "Change the Point

Color” and study the code

- i) Change the code to use 8 different colors. Instead of having a one color per quadrant conceptually divide the canvas up into 8 equal sized “octants” by using 2 additional diagonal lines, at + and - 45 degrees as shown below:



A point in each octant will have a different one of the 8 colors.

- ii) **Git** add and commit with message:

```
"-exercise - ColoredPoints"
```

6) Read the first few sections of Chapter 3.

a) Exercise:

- i) Copy the following sample code from ch03 to your T4_WebGL directory:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp WebGL_Programming_Guide/ch03/MultiPoint.* T4_webGL/
```

- ii) **Git** add and commit with message:

```
"-exercise MultiPoint"
```

- iii) Open `MultiPoint.html` in your browser read Chapter 3 through and including Section “Draw Multiple Points” and read up to and including sub-section “Sample Program (MultiPoint.js)”. Study the code.
- iv) Continue reading and studying the example code until reaching sub-section “Experimenting with the Sample Program”. Now, perform the experiments suggested and save the code change.
- v) **Git** add and commit with message:

```
"-exercise MultiPoint Experiment"
```

7) Exercise:

- a) Copy the following sample code from ch03 to your T4_WebGL directory:

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp WebGL_Programming_Guide/ch03/HelloTriangle.* T4_webGL/
```

- b) **Git** add and commit with message:

"-add Triangles"

- c) Read Section “Hello Triangle” and it’s subsection “Experimenting with the Sample Program”. Make some of the modifications discussed.
- d) **Git** add and commit with message:

"-exercise - Triangles Experiment"

8) Exercise:

- a) Copy the following sample code from ch03 to your T4_WebGL directory with a new file name.

```
zwartell@CCIWD-435B-1 ~/ITCS_3120
$ cp WebGL_Programming_Guide/ch03/HelloTriangle.js
T4_webGL/HelloTrianglePlay.js
$ cp WebGL_Programming_Guide/ch03/HelloTriangle.html T4_webGL/
HelloTrianglePlay.html
```

- b) **Git** add and commit with message:

"-added HelloTrianglePlay"

- c) Continue reading Chapter 3 but stop before reading “Moving, Rotating, and Scaling”
- d) Modify HelloTrianglePlay to include all of the following:
- i) Modify the fragment shader to use a uniform variable to control the color
 - ii) render two TRIANGLE’s of different colors
 - iii) render 1 TRIANGLE_FAN that displays a hexagon (choose any color)
 - iv) render 1 TRIANGLE_STRIP with 5 triangles (choose any color)
 - v) render 1 LINE_STRIP whose edges draw on top of the TRIANGLE_STRIP triangle edges but in a different color
 - vi) Be mildly creative...

- e) **Git** add and commit with message:

"-exercise - Triangle Play"

- 9) Finally Git push your assignment to your 3120_T4_WebGL remote repo. Refer to the “Submitting Class Assignments” section of the Git Tutorial PDF regarding best practices and requirements for final submissions.

5. Academic Integrity

This is an individual assignment. Each student must write and submit their own unique code.

Review the class syllabus regarding the basic requirements for getting full and partial credit for your programming assignments.

Cheating in any form is subject to disciplinary action (UNCC Catalog, pages 275-278).

- As far as programming projects are concerned, you are allowed to discuss general concepts and strategies for solving problems.
- No sharing of modules or parts of programs written by other students is allowed.

The TA will be running the submitted code through various plagiarism detection software such as Turnitin.